

EXAMEN FINAL

Rapport d'évaluation de sécurité

Application web OWASP Juice Shop

Module	Sécurité des Données
Formation	Licence 3 — Cybersécurité
Étudiant	Fatou Bintou Gaye
Rôle	Cybersecurity Analyst / Junior DevSecOps Engineer
Application	OWASP Juice Shop v20.2.0 (bkimminich/juice-shop)
Dépôt	github.com/fatoubinetougaye-dot/examen-secd-data-juiceshop
Décision	Reject Deployment

Sommaire

1.	Introduction	3
2.	Méthodologie	3
3.	Vulnérabilités identifiées (Partie 1)	4
4.	Classification CWE et analyse d'impact CIA (Partie 2)	5
5.	Remédiations et vérifications (Partie 3)	6
6.	Pipeline de sécurité automatisé (Partie 4)	8
7.	Résultats et décision de déploiement (Partie 5)	9
8.	Analyse critique (Partie 6)	10
9.	Conclusion	11

Avertissement

OWASP Juice Shop est une application volontairement vulnérable, publiée par l'OWASP à des fins pédagogiques. Toutes les attaques décrites dans ce rapport ont été réalisées en environnement local isolé (conteneur Docker sur poste personnel), sans jamais viser un système tiers. Ce cadre correspond à la mission d'évaluation confiée dans le scénario de l'examen.

1. Introduction

Une organisation prévoit de mettre en production une application web permettant à ses utilisateurs de créer un compte, de s'authentifier et d'accéder à des ressources contenant des données potentiellement sensibles. Avant ce déploiement, l'équipe de sécurité m'a confié, en tant que *Cybersecurity Analyst / Junior DevSecOps Engineer*, la mission d'évaluer la sécurité de l'application, d'identifier les vulnérabilités susceptibles de compromettre la protection des données, d'en analyser les impacts, de proposer des remédiations et d'intégrer des contrôles de sécurité dans un pipeline automatisé.

L'application retenue comme cible est **OWASP Juice Shop**, une boutique en ligne écrite en Node.js / Express côté serveur et Angular côté client, avec une base de données SQLite via l'ORM Sequelize. Elle reproduit fidèlement l'architecture d'une application réelle tout en concentrant l'ensemble du Top 10 OWASP, ce qui en fait un support d'évaluation représentatif.

Ce rapport couvre les six parties de l'examen : identification des vulnérabilités, analyse de risque selon le triptyque Confidentialité-Intégrité-Disponibilité (CIA), remédiation et vérification, automatisation via un pipeline Jenkins, analyse des résultats et décision de déploiement, puis analyse critique de la démarche outillée.

Objectif de protection des données

Le fil conducteur de l'évaluation est la protection des données des utilisateurs : identifiants de connexion, mots de passe, contenu des paniers et données personnelles. Chaque vulnérabilité est donc analysée à travers le prisme de son effet réel sur ces données plutôt que comme une faiblesse technique isolée.

2. Méthodologie

L'évaluation combine une approche manuelle et une approche outillée, afin de bénéficier de la couverture large des outils automatiques et de la profondeur du test manuel sur la logique métier.

Démarche en cinq temps

- **Reconnaissance** — déploiement de l'application en conteneur Docker et cartographie des points d'entrée (authentification, recherche, panier, serveur de fichiers, API REST).
- **Analyse statique (SAST)** — lecture du code source du dépôt forké et passage de Semgrep avec des règles standard et des règles personnalisées.
- **Analyse de composition (SCA)** — inventaire des dépendances vulnérables via npm audit et Trivy.
- **Analyse dynamique (DAST) et test manuel** — exploitation effective des failles sur l'application en fonctionnement, complétée par un scan OWASP ZAP.
- **Détection de secrets** — recherche de clés et identifiants exposés dans le code et l'historique Git avec Gitleaks.

Référentiels utilisés

Les vulnérabilités sont classées selon la nomenclature **CWE** (Common Weakness Enumeration) et rattachées aux catégories du **Top 10 OWASP 2021**. La criticité est évaluée qualitativement sur quatre niveaux (Critical, High, Medium, Low) en croisant la facilité d'exploitation et l'impact sur les données, selon une logique proche du scoring CVSS.

Environnement de test

Composant	Version
Système d'exploitation	Windows 11 Professionnel (24H2)
Conteneurisation	Docker Desktop 29.4.2
Serveur d'intégration	Jenkins 2.568.2 (service Windows), JDK 21
Runtime applicatif	Node.js 24.19.0
Application cible	github.com/juice-shop/juice-shop (clonée dans app/)

3. Vulnérabilités identifiées

Partie 1 — /4 points

L'évaluation a mis en évidence huit vulnérabilités (le sujet en demande cinq au minimum). Le tableau ci-dessous en donne la description, le composant concerné, la catégorie CWE et le risque pour l'application.

ID	Vulnérabilité	Composant	CWE	Description / risque
V1	Injection SQL	Authentification (routes/login)	CWE-89	La requête de connexion concatène l'e-mail saisi. Le payload ' OR 1=1-- contourne l'authentification et ouvre une session administrateur ; toute la table Users est exfiltrable.
V2	XSS DOM	Barre de recherche (frontend)	CWE-79	Le terme de recherche est passé à bypassSecurityTrustHtml puis injecté en innerHTML. Du code JavaScript arbitraire s'exécute dans le navigateur de la victime (vol de jeton, actions frauduleuses).
V3	Broken Access Control / IDOR	Panier (routes/basket)	CWE-639	L'identifiant de panier vient de l'URL sans contrôle d'appartenance. Un utilisateur authentifié lit et modifie le panier d'autrui par simple changement d'ID.
V4	Hachage de mot de passe faible	lib/insecurity	CWE-916	Les mots de passe sont hachés en MD5 sans sel. Un vol de base est cassé en quelques minutes (rainbow tables, hashcat).
V5	Exposition de données sensibles	Serveur de fichiers (/ftp)	CWE-200	Le répertoire /ftp est listable et sert des sauvegardes. Combiné à une traversée par octet nul (%2500), il divulgue des fichiers confidentiels.
V6	Secrets codés en dur	lib/insecurity	CWE-798	La clé privée RSA de signature des JWT est embarquée dans le code. Un attaquant peut forger des jetons valides pour n'importe quel compte.
V7	Dépendances vulnérables	package.json	CWE-1395	npm audit remonte 45 vulnérabilités (7 critiques, 19 élevées), dont pollution de prototype (lodash), injection de commande (marsdb) et traversée d'archive Zip Slip (decompress).
V8	Mauvaise configuration de sécurité	server.ts (en-têtes HTTP)	CWE-16	Absence de CSP, HSTS, X-Frame-Options, X-Content-Type-Options ; en-tête X-Powered-By exposé. Surface d'attaque XSS et clickjacking élargie.

Les vulnérabilités V1 à V4 sont retenues pour la phase de remédiation détaillée (Partie 3), car elles combinent une exploitation directe et un impact maximal sur les données des utilisateurs.

4. Classification CWE et analyse d'impact CIA

Partie 2 — /4 points

Chaque vulnérabilité est analysée selon les trois propriétés fondamentales de la sécurité de l'information — Confidentialité, Intégrité, Disponibilité — puis se voit attribuer un niveau de criticité et une priorité de correction.

Vuln.	Confid.	Intég.	Dispo.	Criticité	Priorité de correction
V1 SQLi	Oui	Oui	Possible	Critical	Immédiate — contournement d'authentification, fuite totale de la base.
V2 XSS	Oui	Possible	Non	High	Élevée — vol de session et actions au nom de la victime.
V3 IDOR	Oui	Oui	Non	High	Élevée — accès et modification des données d'autres utilisateurs.
V4 MD5	Oui	Possible	Non	Critical	Immédiate — compromet tous les comptes en cas de fuite de base.
V5 /ftp	Oui	Non	Non	Medium	Planifiée — divulgation de fichiers, sans écriture.
V6 Secret	Oui	Oui	Non	Critical	Immédiate — forge de jetons, usurpation de tout compte.
V7 Deps	Oui	Possible	Possible	High	Élevée — dépend du chemin d'appel, correctifs à appliquer.
V8 Config	Possible	Non	Non	Medium	Planifiée — durcissement des en-têtes HTTP.

Lecture des priorités

Les vulnérabilités critiques (V1, V4, V6) partagent une caractéristique : elles ouvrent un accès aux données de l'ensemble des utilisateurs, pas seulement de la victime directe. Elles sont donc traitées en priorité absolue. Les vulnérabilités élevées (V2, V3, V7) portent une atteinte sérieuse mais plus circonscrite. Les vulnérabilités moyennes (V5, V8) relèvent du durcissement et sont planifiées une fois les précédentes corrigées.

Concernant la disponibilité, l'injection SQL est marquée « Possible » car une requête malveillante (par exemple une clause coûteuse ou une commande de suppression) peut dégrader ou rendre indisponible le service, au-delà de la seule lecture de données.

5. Remédiations et vérifications

Partie 3 — /4 points

Quatre vulnérabilités importantes sont corrigées ci-dessous (le sujet en demande trois). Pour chacune : la cause, la correction proposée, la justification de son efficacité et la méthode de vérification. Le code avant/après complet figure dans le dossier *remediation/* du dépôt Git.

V1 — Injection SQL (CWE-89, Critical)

Cause	La requête de connexion est construite par interpolation de chaîne : l'e-mail saisi devient une partie de la structure SQL exécutée.
Correction	Passage à une requête paramétrée Sequelize (option <code>replacements</code>), la valeur utilisateur devenant une donnée liée et non du code. Ajout d'une validation d'entrée (format e-mail, longueur bornée) et d'un message d'erreur générique.
Justification	Le driver échappe la donnée liée : la clause <code>1=1</code> est traitée comme une chaîne littérale et ne modifie plus la logique. La donnée ne peut structurellement plus s'exécuter comme du SQL.
Vérification	Le payload <code>' OR 1=1--</code> renvoie désormais 401 au lieu de 200 ; une connexion légitime fonctionne toujours ; Semgrep et sqlmap ne signalent plus la faille. Un test automatisé verrouille la correction.

V2 — XSS DOM (CWE-79, High)

Cause	Le terme de recherche est marqué de confiance via <code>bypassSecurityTrustHtml</code> , ce qui neutralise le sanitizer Angular, puis injecté en <code>innerHTML</code> .
Correction	Suppression de l'appel <code>bypassSecurityTrustHtml</code> et remplacement de <code>[innerHTML]</code> par l'interpolation <code>{{ }}</code> , qui applique l'encodage HTML automatique. Ajout d'une <code>Content-Security-Policy</code> stricte via <code>Helmet</code> en défense en profondeur.
Justification	L'encodage contextuel transforme les caractères actifs (<code><</code> <code>></code> <code>"</code> <code>'</code>) en entités inertes : le navigateur affiche le texte au lieu de l'exécuter. La CSP bloque en dernier recours tout script en ligne.
Vérification	Les charges XSS s'affichent littéralement ; l'inspection du DOM montre des entités HTML et non des nœuds actifs ; ZAP ne remonte plus d'alerte XSS ni d'absence de CSP.

V3 — Broken Access Control / IDOR (CWE-639, High)

Cause	L'identifiant de panier provient de l'URL sans vérification que la ressource appartient à l'utilisateur authentifié.
Correction	L'identité est désormais lue exclusivement dans le jeton JWT vérifié côté serveur. Le filtre d'appartenance (<code>UserId</code>) est intégré directement à la requête, et un accès non autorisé renvoie 404. La tentative est journalisée.
Justification	Le contrôle d'accès est appliqué au niveau de la requête (fail-safe) : un panier tiers ne peut plus être retourné, même en cas d'erreur applicative. L'identité n'étant plus dérivée d'un paramètre client, elle n'est plus falsifiable.
Vérification	Un scénario à deux comptes montre 404 sur le panier d'autrui et 200 sur le sien ; l'énumération de 1 à 20 ne retourne plus qu'un seul panier ; chaque refus produit une trace exploitable par un SIEM.

V4 — Hachage de mot de passe faible (CWE-916, Critical)

Cause	Les mots de passe sont dérivés en MD5, fonction rapide et sans sel, donc vulnérable aux rainbow tables et au cassage massif.
Correction	Remplacement par bcrypt avec un facteur de coût 12 (sel unique automatique, dérivation lente), comparaison à temps constant, politique de mot de passe minimale et migration transparente des comptes existants au premier login.
Justification	bcrypt ralentit chaque essai d'un facteur considérable et rend chaque haché unique grâce au sel : une attaque par dictionnaire complète devient économiquement irréaliste sur un mot de passe conforme.
Vérification	Le format stocké passe de 32 hex à un haché \$2a\$12\$; deux comptes de même mot de passe ont des hachés différents ; hashcat chute de milliards à quelques milliers d'essais/s ; Semgrep ne signale plus MD5.

6. Pipeline de sécurité automatisé

Partie 4 — /4 points

Un pipeline Jenkins déclaratif (fichier *Jenkinsfile* du dépôt) automatise les contrôles de sécurité à chaque évolution du code. Il respecte les six étapes imposées et intègre quatre types de contrôles (le sujet en demande deux) : SAST, SCA, Secret Detection et DAST.

Enchaînement des étapes

Étape	Contenu
1. Checkout	Récupération du code depuis GitHub (credential github-cred).
2. Build / Preparation	Installation des dépendances et démarrage du conteneur Juice Shop (cible du DAST).
3. Security Analysis	SAST (Semgrep) et SCA (npm audit + Trivy) exécutés en parallèle.
4. Additional Security Check	Secret Detection (Gitleaks) et DAST (OWASP ZAP) exécutés en parallèle.
5. Report Generation	Consolidation des rapports en un résumé HTML/JSON, archivage, application du quality gate.
6. Notification	Envoi du bilan et de la décision par e-mail (Email Extension), nettoyage de l'environnement.

Outils intégrés

Outil	Type	Détecte	Limites	Moment
Semgrep	SAST	Injections, XSS, crypto faible, secrets, motifs dangereux dans le code	Aveugle au runtime ; faux positifs ; rate la logique métier	Étape 3
npm audit	SCA	CVE des dépendances npm et transitives	N'indique pas si le code vulnérable est appelé	Étape 3
Trivy	SCA / config	CVE OS et librairies, mauvaises configurations	Bruit sur paquets non exposés	Étape 3
Gitleaks	Secret Detection	Clés, jetons, identifiants dans le code et l'historique Git	Ne valide pas le secret ; faux positifs sur les jeux de test	Étape 4
OWASP ZAP	DAST	XSS, injections, en-têtes manquants, cookies non sécurisés à l'exécution	Baseline passif : ne teste pas le périmètre authentifié ni la logique	Étape 4

Le quality gate applique une politique explicite : zéro vulnérabilité critique et au plus cinq vulnérabilités élevées. Au-delà, le build échoue et le déploiement est bloqué. La décision est ainsi automatisée et reproductible plutôt que laissée à l'appréciation ponctuelle d'un opérateur.

7. Résultats et décision de déploiement

Partie 5 — /2 points

L'exécution du pipeline consolide les résultats des différents outils. Le tableau ci-dessous priorise les problèmes détectés et associe à chacun l'action recommandée.

Problème	CWE	Sévérité	Action recommandée
Injection SQL	CWE-89	Critical	Bloquer le déploiement
Hachage MD5 des mots de passe	CWE-916	Critical	Bloquer le déploiement
Clé privée codée en dur	CWE-798	Critical	Bloquer le déploiement
XSS DOM	CWE-79	High	Correction requise
Broken Access Control (IDOR)	CWE-639	High	Correction requise
Dépendances vulnérables	CWE-1395	High	Correction requise
Exposition d'information (/ftp)	CWE-200	Medium	Correction planifiée
En-têtes de sécurité absents	CWE-16	Medium	Correction planifiée

Décision finale

REJECT DEPLOYMENT

Justification

L'application présente au moins trois vulnérabilités critiques exploitables (injection SQL, hachage MD5, clé privée en dur) qui touchent directement la confidentialité et l'intégrité des données de tous les utilisateurs. Le seuil de la politique (zéro critique) est franchi. Déployer en l'état exposerait immédiatement les comptes et les données personnelles. La mise en production est donc refusée : les vulnérabilités critiques et élevées doivent être corrigées, puis le pipeline ré-exécuté pour confirmer le retour sous les seuils avant toute nouvelle décision.

Il ne s'agit pas d'un « Accept with Conditions » : les défauts constatés ne sont pas des points de durcissement mineurs mais des failles activement exploitables menant à une compromission complète.

8. Analyse critique

Partie 6 — /2 points

Un outil de sécurité qui ne détecte aucune vulnérabilité peut-il garantir qu'une application est sécurisée ?

Non. L'absence de détection ne prouve pas l'absence de vulnérabilité ; elle prouve seulement que les règles configurées n'ont rien trouvé dans le périmètre analysé. Confondre les deux est une erreur d'interprétation aux conséquences graves.

Faux positifs et faux négatifs

Un outil produit des **faux positifs** (alertes sur du code sain) qui, trop nombreux, noient les vraies alertes et poussent les équipes à désactiver les contrôles. Il produit surtout des **faux négatifs** (vulnérabilités réelles non signalées), bien plus dangereux car invisibles : ils installent une fausse confiance. Dans ce projet, la faille IDOR (V3) illustre ce risque — un contrôle d'accès défaillant relève de la logique métier et échappe largement à la détection purement syntaxique.

Limites des outils et couverture des règles

Chaque outil a un angle mort. Le SAST ignore le comportement à l'exécution ; le DAST en mode baseline ne teste pas le périmètre authentifié ni les enchaînements métier ; le SCA signale des CVE sans savoir si le code vulnérable est réellement atteint ; la détection de secrets ne juge pas de la validité d'un secret. La couverture dépend directement des règles activées : une catégorie de faille sans règle correspondante restera invisible, quel que soit le nombre de scans. Les outils ne voient bien que ce qu'on leur a appris à chercher.

Importance de l'analyse humaine

L'automatisation est indispensable pour passer à l'échelle et détecter tôt les régressions, mais elle ne remplace pas le jugement. L'analyste humain contextualise une alerte selon la sensibilité réelle des données, écarte les faux positifs, découvre les failles de logique métier et tranche la décision de déploiement. Les outils et l'humain sont complémentaires : les premiers élargissent la couverture et accélèrent, le second apporte la compréhension du contexte et du risque.

En synthèse, un scan « propre » est une condition nécessaire mais non suffisante : il réduit le risque connu, il ne certifie jamais l'absence de risque. La sécurité se démontre par un faisceau de preuves convergentes — outils variés, tests manuels et revue humaine — et non par le silence d'un seul outil.

9. Conclusion

L'évaluation d'OWASP Juice Shop a mis en évidence huit vulnérabilités, dont trois critiques touchant directement la protection des données des utilisateurs. Quatre d'entre elles ont été corrigées et les corrections vérifiées par rejeu d'attaque, test de non-régression et confirmation outillée.

Un pipeline Jenkins intégrant SAST, SCA, détection de secrets et DAST a été mis en place pour automatiser ces contrôles et matérialiser la décision de déploiement via un quality gate explicite. Appliqué à l'application en l'état, ce pipeline aboutit logiquement à un **Reject Deployment**.

Au-delà du verdict, ce travail illustre une posture DevSecOps : intégrer la sécurité au plus tôt et de façon continue, tout en gardant à l'esprit que l'outillage n'a de valeur qu'accompagné d'une analyse humaine capable d'en interpréter les résultats et d'en combler les angles morts.